CaseVault | Phase 01

Project selection & architecture baseline

STATUS: Proposed — awaiting instructor approval. This document is a design baseline, not a claim of completed security implementation.

1. Selection decision

Recommended: CaseVault — a secure incident-evidence exchange platform built as an internal web application. It is more domain-specific than a generic file-sharing site, while naturally supporting all five mandatory demonstration classes and a feasible seven-day build.

Candidate	Security-class fit	Time-box risk	Decision
Generic CMS/blog	Good, but native parser often feels bolted on	Medium	Not selected
CI/CD build portal	Natural command injection; complicated jobs and permissions	High	Not selected
CaseVault evidence portal	Natural IDOR/CSRF/framing and external file helper; native legacy metadata processor	Medium / controlled	Recommended

2. Problem / users / product promise

Problem. Teams handling sensitive incident evidence need private transfers, accountable sharing, and a way to verify file integrity; insecure reference handling or file processing can expose their data.

Users. Evidence owner (investigator), authorised collaborator (analyst), operator (maintains service). Attackers are modelled as external or authenticated adversaries, not product roles.

Value. Authenticated evidence exchange, per-object access control, SHA-256 checksums, an append-only-at-application-level audit record, and repeatable automated security verification.

3. Project scope (minimum usable product)

- Register / sign in / sign out; hashed passwords and session controls.
- Upload a bounded-size evidence file with safe metadata validation; generate SHA-256 digest and opaque internal storage ID.
- List own and explicitly shared evidence; owner grants/revokes access by registered username.
- Download only after server-side authorisation; owner can delete evidence.
- Record important actions (upload/share/revoke/download/delete) in the audit table.
- Expose a health endpoint and launch locally through Docker Compose.

Not in sprint scope: live malware processing, public download links, enterprise SSO, distributed storage, full incident case management, real-time chat, or claims of regulatory compliance.

4. High-level system architecture

This architecture is the Phase 01 design sketch. Formal context and Level-1 DFDs with trust boundaries will be created in Phase 03.

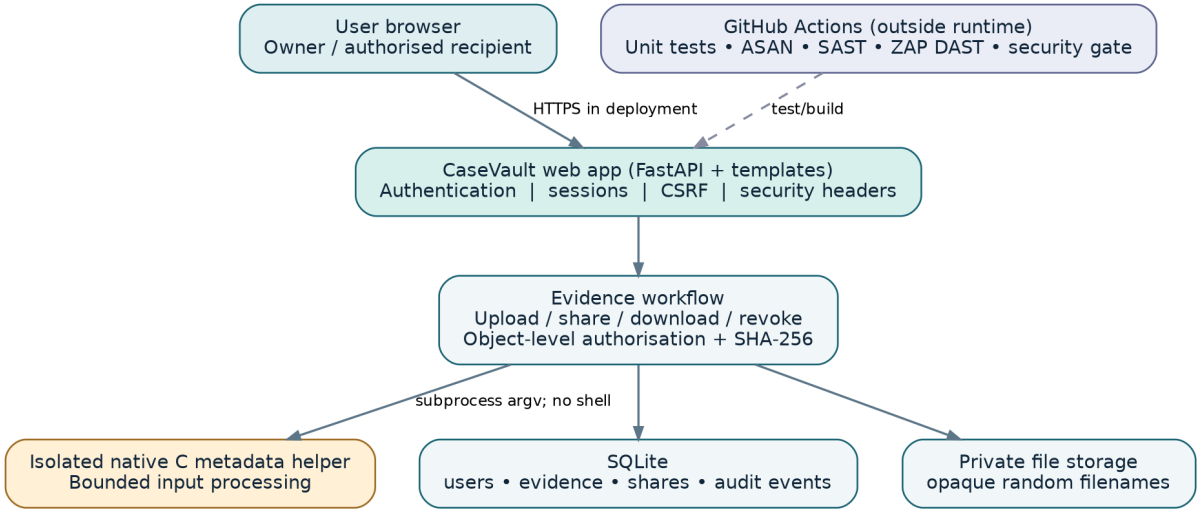


Figure 1 — Runtime architecture and external CI verification path. Dashed arrow indicates build/test only.

5. Technology decisions and why

Layer	Selected tool	Reason
Web interface	FastAPI + Jinja2 HTML	Lightweight authenticated browser workflows; no SPA complexity
Data	SQLite + private local storage	Simple reproducible lab demo; secret files not in a public static directory
Native boundary	Small C metadata helper invoked as a subprocess	Real upload path; controlled legacy memory-safety exercise
Integrity	SHA-256 in Python hashlib	Avoid using external shell tools for digest creation
Deployment	Docker Compose	One-command local reproducibility
Security testing	pytest / ASAN / Semgrep or CodeQL / ZAP	Repeatable unit, memory, static and dynamic testing
Collaboration	GitHub PRs + Jira board	Issue→threat→fix→test traceability

6. Mandatory vulnerability feasibility matrix

Each lab weakness must be exhibited in a local, controlled environment and shown to be blocked by a genuine code change or defensive control. “Expected proof” refers to evidence to capture later, not completed results.

Class	Relevant application surface	Before / after proof to collect
CWE-120 Buffer Overflow	Legacy native C evidence-filename/metadata parsing	ASAN failure on overlong input; bounded secure parser exits safely
CWE-78 Command Injection	Legacy shell command invoking file helper using user-controlled metadata	Marker/injection possible in legacy lab; secure argv invocation treats metacharacters as data
CWE-639 IDOR	Evidence download/share/revoke routes with predictable IDs	Owner vs second account; unauthorised object rejected consistently
CWE-352 CSRF	Share/revoke/delete authenticated browser POSTs	Missing/invalid token rejected; valid token succeeds
CWE-1021 Clickjacking	Authenticated dashboard/share page	Vulnerable lab page framed; secure page refuses embedding in browser

7. Security boundaries and design rules

Browser → server. Every request is untrusted. Sensitive state changes require an authenticated session and CSRF validation.

Application → database. Authorise the exact evidence object on each operation; use parameterised SQL; keep DB private.

Application → native helper. Pass a fixed executable plus validated argument vector; apply strict length limits and subprocess timeout.

Application → file store. Never trust client paths; use generated opaque filenames; limit upload size and resource usage.

Deployment / supply chain. Keep secrets out of Git, run as non-root, scan dependencies, and configure HTTPS before internet exposure.

8. Product-level CIA outcomes

CIA	Proposed control / expected verifiable outcome
Confidentiality	Unshared evidence cannot be retrieved using another account or guessed object ID.
Integrity	Evidence bytes have a recorded SHA-256 digest; actions require authorised state changes.
Availability	Uploads are size-bounded; native helper times out; health check and repeatable container launch.

9. Seven-day implementation boundary

Timing	Phase	Output / approval gate
Day 0–1	Theme + repo setup; PNE	Approved proposal, repository, working Jira board, initial PNE/CIA inventory
Day 1–2	Threat model	Context + Level-1 DFDs, STRIDE, two Attack Trees, risk register with CVSS vectors
Day 2–5	Build and secure	Complete browser workflow plus all five lab before/after demonstrations and fixes
Day 5–6	Automate and scan	Build/tests/SAST and DAST; high-or-critical finding must fail security check
Day 6–7	Demonstrate and submit	18–30-page report, 6–10 min video, final GitHub/Jira evidence

10. Risks, decisions, and honest limitations

Approval and uniqueness. Must be confirmed by instructor before locking theme; no search can prove uniqueness within the class.

Native C credibility. Native helper must be executed in the upload flow; a disconnected “hello-world overflow” is insufficient.

Legacy lab safety. Keep vulnerable binaries/routes out of the released app and container; only reproduce with dummy data and localhost.

Local versus internet production. SQLite/local storage and non-TLS localhost are acceptable for a demo, but not equivalent to production readiness; document TLS, backups, rotation, malware scanning, and audit integrity as limitations/next steps.

Real security evidence. Screenshots, code-review approvals, CI runs and ZAP artifacts must be generated during authentic team work, not claimed in advance.

11. Phase 01 acceptance checklist

☐ Instructor receives the half-page proposal within the first 24 hours of the actual sprint window.

☐ Instructor confirms theme uniqueness / approves (or requests changes).

☐ Team members and actual date are completed on the proposal.

☐ Team agrees on product scope, tech stack and safe-lab boundary.

☐ Approval submission screenshot and response are saved for the final evidence folder.

Official design references

CYC386 Midterm Lab Exam brief, Sections 2–6; OWASP CSRF Prevention and Clickjacking Defense Cheat Sheets; MITRE CWE-120. These references guide the planned controls; no implementation results are asserted in this blueprint.